

Day 2: Core JavaScript Study Notes

In-Depth Reference Guide: Scope, Closures, Higher-Order Functions & Object Context (`this`)

Day 2: Comprehensive JavaScript Study Notes

Table of Contents

1. Scope in JavaScript

- Global Scope
- Function Scope
- Block Scope & `var` vs `let` vs `const`
- Variable Shadowing
- The Scope Chain & Lookup Process

2. Lexical Scope

- What "Lexical" Really Means
- The One-Way Access Rule

3. Closures

- Definition & Mechanics
- Memory Model (Call Stack vs Heap)
- Common Use Cases

4. Higher-Order Functions (HOFs) & Callbacks

- First-Class Functions
- Passing Functions as Arguments (Callbacks)
- Returning Functions from Functions (Function Factories)
- Building Custom Iterators (`myFilter`)

5. Objects and the `this` Keyword

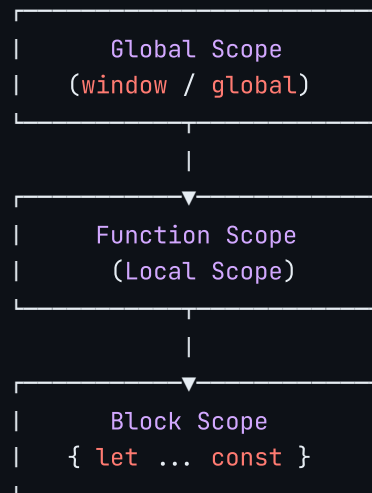
- What is `this` ?

- Method Invocation Binding
- Why `this.property` vs `object.property` ?
- Arrow Functions vs Regular Functions with `this`

6. Day 2 Practice Exercises: Deep Dive & Solutions

1. Scope in JavaScript

Scope determines the **accessibility (visibility)** of variables, objects, and functions from different parts of your code. In JavaScript, scope defines the boundaries where a variable is declared and where it can be used.



Global Scope

Any variable declared outside of all functions or blocks belongs to the **Global Scope**.

- Accessible anywhere in the script.
- Lives in memory for the entire duration the web page or Node.js process runs.
- **Polluting the global scope** can lead to naming collisions and hard-to-track bugs.

```
let globalVar = "I am accessible anywhere";

function showGlobal() {
  console.log(globalVar); // Accessible!
}

showGlobal();
```

Function Scope

Every function creates its own **execution context** and local scope.

- Variables declared inside a function are local to that function.
- They are created when the function is called and cleaned up by garbage collection after the function finishes execution.

```
function test() {  
    let localVar = "Secret";  
    console.log(localVar); // Works: "Secret"  
}  
test();  
console.log(localVar); // ReferenceError: localVar is not defined
```

Block Scope & `var` vs `let` vs `const`

Prior to ES6 (2015), JavaScript only had Global and Function scope (`var`). ES6 introduced `let` and `const`, which honor **Block Scope**.

A block is delimited by curly braces `{ ... }` (e.g., inside `if`, `switch`, `for`, `while`, or standalone blocks).

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function scoped	Block scoped	Block scoped
Hoisting	Hoisted (initialized as <code>undefined</code>)	Hoisted (in Temporal Dead Zone)	Hoisted (in Temporal Dead Zone)
Re-declaration	Allowed in the same scope	SyntaxError	SyntaxError
Re-assignment	Allowed	Allowed	TypeError (immutable binding)

Block Scope Example:

```
if (true) {  
  var a = 10;  
  let b = 20;  
  const c = 30;  
}  
  
console.log(a); // 10 (var leaks out of blocks!)  
console.log(b); // ReferenceError: b is not defined  
console.log(c); // ReferenceError: c is not defined
```

Important: When attempting to access an out-of-scope `let` or `const` variable, JavaScript throws a `ReferenceError`, NOT a `TypeError`.

Variable Shadowing

Shadowing occurs when a variable declared within a local/inner scope has the **same name** as a variable in an outer scope.

```
let name = "Dibyo"; // Outer scope  
  
function greet() {  
  let name = "John"; // Shadows the outer "name"  
  console.log(name); // Prints: "John"  
}  
  
greet();  
console.log(name); // Prints: "Dibyo" (Outer variable was untouched)
```

How Shadowing Works:

1. The JavaScript engine starts resolving variable identifiers starting from the **innermost** scope.
2. The moment it finds a variable matching the requested name, it uses it immediately and halts any further search up the chain.
3. The outer variable is "shadowed" (hidden) for that execution context, but remains intact outside.

The Scope Chain & Lookup Process

When the engine needs a variable:

1. Searches the current local scope.

2. If not found, steps up to the parent/enclosing scope.
3. Continues outward until it reaches the global scope.
4. If still not found in the global scope:
 - Reading: throws `ReferenceError`.
 - Writing (in non-strict mode with `var` behavior): creates an accidental global.

2. Lexical Scope

What "Lexical" Really Means

Lexical Scope (also called Static Scope) means that variable resolution is determined by the **physical placement** of functions in the source code at author time, **not** where or how functions are called at runtime.

```
let x = 10;

function outer() {
  let y = 20;

  function inner() {
    // inner() has lexical access to both x and y
    console.log(x); // Found in Global Scope: 10
    console.log(y); // Found in Outer Scope: 20
  }

  inner();
}

outer();
```

The One-Way Access Rule

```
[ Outer Scope / Parent ]
|
| ▲ Inner CAN look UP into Parent
| |
| ▼ X Parent CANNOT look DOWN into Inner
[ Inner Scope / Child ]
```

1. **Child scopes can look outward:** An inner function can access variables declared in its enclosing parent function and the global environment.

2. **Parent scopes cannot look inward:** An outer function cannot reach inside a nested function to read its local variables.

```
function outer() {
  function inner() {
    let secret = 100;
  }
  inner();
  console.log(secret); // ReferenceError: secret is not defined
}
outer();
```

3. Closures

Definition & Mechanics

A **Closure** is the combination of a function bundled together with references to its surrounding state (**lexical environment**).

In plain terms: **A closure gives an inner function access to an outer function's scope even after the outer function has finished executing.**

```
function outer() {
  let counter = 0;

  function increment() {
    counter++;
    return counter;
  }

  return increment;
}

const count = outer(); // outer() runs and finishes
console.log(count()); // 1
console.log(count()); // 2
console.log(count()); // 3
```

Memory Model (Call Stack vs Heap)

Normally, when a function finishes executing, its execution context is popped off the **Call Stack** and its local variables are eligible for garbage collection.

However, when an inner function is returned (or passed elsewhere) and maintains a reference to variables from the outer function:

- The JavaScript engine moves those referenced variables from the stack to the **Heap**.
- The closure retains a live reference to that memory space.

Common Use Cases

1. Data Encapsulation / Private State:

```
function createBankAccount(initialBalance) {  
  let balance = initialBalance; // Private variable  
  
  return {  
    deposit(amount) { balance += amount; return balance; },  
    withdraw(amount) { balance -= amount; return balance; },  
    getBalance() { return balance; }  
  };  
}  
  
const account = createBankAccount(500);  
account.deposit(100);  
console.log(account.getBalance()); // 600  
console.log(account.balance); // undefined (cannot be tampered with!)
```

2. Function Factories: Generating functions customized with initial configurations (e.g., multiplier, threshold checker).

4. Higher-Order Functions (HOFs) & Callbacks

First-Class Functions

In JavaScript, functions are **first-class citizens** (or first-class objects). This means:

- Functions can be assigned to variables or object properties.
- Functions can be passed as arguments to other functions.
- Functions can be returned from other functions.

Passing Functions as Arguments (Callbacks)

A **Higher-Order Function** is any function that:

1. Takes one or more functions as arguments, **OR**

2. Returns a function as its result.

A function passed into another function as an argument is called a **Callback**.

```
// HOF taking a callback
function repeatTask(task, n) {
  for (let i = 0; i < n; i++) {
    task(i);
  }
}

repeatTask(function(index) {
  console.log(`Executing step #${index + 1}`);
}, 3);
```

Decoupling Logic with Strategy Callbacks:

```
function calculate(a, b, operation) {
  return operation(a, b);
}

const add = (x, y) => x + y;
const multiply = (x, y) => x * y;

console.log(calculate(10, 5, add));      // 15
console.log(calculate(10, 5, multiply)); // 50
```

Returning Functions from Functions (Function Factories)

A function factory builds and returns a specialized function using closures to remember its initialization parameters:


```
// Factory Function
function greaterThan(threshold) {
  return function(number) {
    return number > threshold;
  };
}

const greaterThan10 = greaterThan(10);
console.log(greaterThan10(15)); // true
console.log(greaterThan10(7));  // false

const isAdult = greaterThan(18);
console.log(isAdult(20)); // true
console.log(isAdult(16)); // false
```

```
// Number Checker Factory
function checkNumber(type) {
  if (type === "even") {
    return function(num) {
      return num % 2 === 0;
    };
  } else if (type === "odd") {
    return function(num) {
      return num % 2 !== 0;
    };
  }
}

const checkEven = checkNumber("even");
console.log(checkEven(10)); // true
console.log(checkEven(7));  // false
```

Building Custom Iterators (`myFilter`)

Understanding how native array methods work under the hood:

```
function myFilter(arr, callback) {
  const filteredArray = [];
  for (let i = 0; i < arr.length; i++) {
    // Pass (currentElement, currentIndex, originalArray)
    if (callback(arr[i], i, arr)) {
      filteredArray.push(arr[i]);
    }
  }
  return filteredArray;
}

const numbers = [1, 2, 3, 4, 5, 6];
const evens = myFilter(numbers, (num) => num % 2 === 0);
console.log(evens); // [2, 4, 6]
```

5. Objects and the `this` Keyword

What is `this` ?

In JavaScript, `this` is a keyword that refers to the **context** in which the current code is being executed.

Unlike variables which follow lexical scoping, the value of `this` in standard functions is determined by **how the function is called** (runtime binding).

How is the `function` invoked?



Method Invocation Binding

When a function is called as a method of an object (`obj.method()`), `this` refers to `obj` (the object before the dot):

```
const person = {
  name: "Diby",
  introduce() {
    console.log(`Hello, my name is ${this.name}`);
  }
};

person.introduce(); // "Hello, my name is Diby" (this === person)
```

Why `this.property` vs `object.property` ?

Why should we use `this.name` inside an object method instead of writing `user.name` ?

1. **Object Renaming & Decoupling:** If you rename the variable holding the object, hardcoded references break:

```
const user = {
  name: "Diby",
  greet() {
    console.log(user.name); // Fragile: tight coupling to variable name
  }
};

const client = user; // Another reference
// If `user` is reassigned or deleted, client.greet() will fail or give stale data!
```

2. **Reusability & Method Sharing:** You can share a single function across multiple objects, and `this` will dynamically adapt to whichever object called it:

```
function saySalary() {
  return this.salary + this.bonus;
}

const emp1 = { salary: 50000, bonus: 5000, getTotal: saySalary };
const emp2 = { salary: 80000, bonus: 12000, getTotal: saySalary };

console.log(emp1.getTotal()); // 55000
console.log(emp2.getTotal()); // 92000
```

Arrow Functions vs Regular Functions with `this`

- **Regular Functions:** Have their own `this` binding determined at call time.
- **Arrow Functions:** Do NOT have their own `this`. They inherit `this` lexically from their enclosing scope at author time.

```
const student = {
  name: "Saif",
  regularMethod: function() {
    console.log(this.name); // "Saif"
  },
  arrowMethod: () => {
    console.log(this.name); // undefined (inherits global window/module this)
  }
};
```

Rule of Thumb: Do not use arrow functions for top-level object methods when you need access to object properties via `this`.

6. Day 2 Practice Exercises: Deep Dive & Solutions

Q1 to Q5: Scope and Shadowing

- **Q1:** `test()` logs `20` because of its local `let x = 20`. `console.log(x)` outside logs `10` because the global variable was never modified.
- **Q2:** `greet()` prints `"John"` and the outer log prints `"Dibyo"`. The local `name` shadowed the global `name`.
- **Q3 / Block Scope:** `let` is block-scoped. Inside `{ let x = 200; }`, `x` is local to those curly braces and ceases to exist outside.
- **Q4:** Accessing `a` or `b` outside `if (true) { let a = 10; const b = 20; }` throws a `ReferenceError: a is not defined`.
- **Q5:**
 - Inside `if` block: `30`
 - Inside `outer()`: `20`
 - Global scope: `10`

Q6 to Q9: Lexical Scoping

- **Q6:** `inner()` accesses `x` from the global scope and `y` from its parent `outer()` scope through lexical scope chaining.
- **Q7:** In `outer()`, `console.log(a)` succeeds (`10`), but `console.log(b)` throws a `ReferenceError` because `b` is scoped inside `inner()`.

- **Q8:** Three levels of `a`: inner function has `a = 3`, `outer()` has `a = 2`, global has `a = 1`. Calling `outer(); console.log(a);` prints `1`.
- **Q9:** Outer functions cannot access variables declared inside child functions because scope lookup is strictly unidirectional (inward cannot see outward, only child $\rightarrow$ parent).

Q10 to Q14: Higher-Order Functions

- **Q10** (`repeatTask`):

```
function repeatTask(task, n) {
  for (let i = 0; i < n; i++) {
    task();
  }
}
```

- **Q11** (`calculate`):

```
function calculate(a, b, operation) {
  return operation(a, b);
}
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
const multiply = (a, b) => a * b;
```

- **Q12 & Q13 (Function Factories):**

```
// Factory returning a predicate function
function greaterThan(n) {
  return function(val) {
    return val > n;
  };
}
```

- **Q14** (`myFilter`):

```
function myFilter(arr, condition) {
  const output = [];
  for (let i = 0; i < arr.length; i++) {
    if (condition(arr[i])) {
      output.push(arr[i]);
    }
  }
  return output;
}
```

Q15 & Q18 to Q20: Objects and `this`

- Q19 (`employee`):

```
const employee = {
  name: "John",
  salary: 50000,
  bonus: 10000,
  getTotalSalary() {
    return this.salary + this.bonus;
  }
};

console.log(employee.getTotalSalary()); // 60000
```

Q24: Student Result System (Capstone)

Combining Objects, Higher-Order Functions, and `this`:

```
const student = {
  name: "Diby",
  marks: [80, 90, 70],

  // Passes this.marks into the provided callback operation
  calculate(operation) {
    return operation(this.marks);
  }
};

// Pure operations receiving an array of marks:
function getTotal(marks) {
  return marks.reduce((sum, mark) => sum + mark, 0);
}

function getAverage(marks) {
  if (marks.length === 0) return 0;
  return getTotal(marks) / marks.length;
}

function getHighest(marks) {
  return Math.max(...marks);
}

// Execution:
console.log("Total:", student.calculate(getTotal)); // 240
console.log("Average:", student.calculate(getAverage)); // 80
console.log("Highest:", student.calculate(getHighest)); // 90
```

Key Takeaways

1. **Scope controls lifetime and visibility:** Use `const` by default, `let` when reassigning, avoid `var`.
2. **Lexical Scope is static:** Scope is defined by code structure at write time.
3. **Closures preserve outer variables:** Essential for encapsulation, stateful helpers, and factory functions.
4. **HOFs enable clean composition:** Pass functions to specialize behavior without repeating control loops.
5. `this` **is bound at invocation:** For standard methods, `this` is the object to the left of the dot.